

Does Zero-Shot Reinforcement Learning Exist?

백승언, 김현성, 이도현 , 정강민

11 June, 2023

- **Introduction**

- Current Success of AI
- Meta Reinforcement Learning

- **Does Zero-Shot Reinforcement Learning Exist?**

- Backgrounds
- Previous Strategies for Zero-Shot RL
- Algorithms for SF and FB Representations

- **Experiments**

- Environments
- Results

Introduction

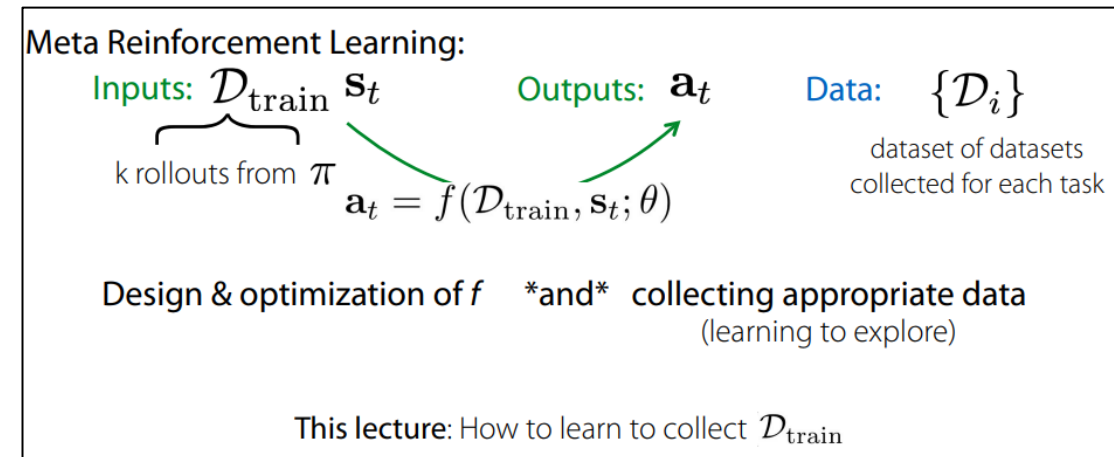
Meta Reinforcement Learning

● Problem setting of reinforcement learning, meta-learning

- Reinforcement learning
 - Given certain MDP, learn a policy π that maximize the expected discounted return $\mathbb{E}_{\pi, p_0} [\sum_{t=0}^{\infty} \gamma^{t-1} r_t(s_t, a_t, s_{t+1})]$
- Meta-learning
 - Given data from $\mathcal{T}_1, \dots, \mathcal{T}_N$, quickly solve new task $\mathcal{T}_{test}$

● Problem setting of Meta-Reinforcement Learning(Meta-RL)

- Setting 1: meta-learning with diverse goal(goal as a task)
 - $\mathcal{T}_i \triangleq \{\mathcal{S}, \mathcal{A}, p(s_0), p(s'|s, a), r(s, a, g), g_i\}$
- Setting 2: meta-learning with RL tasks(MDP as a task)
 - $\mathcal{T}_i \triangleq \{\mathcal{S}_i, \mathcal{A}_i, p_i(s_0), p_i(s'|s, a), r_i(s, a)\}$



Meta RL problem statement in CS-330(Finn)

Does Zero-Shot Reinforcement Learning Exist?

● Notation

■ Reward-free MDP

- $\mathcal{M} = (S, A, P, \gamma)$ is a reward-free Markov Decision Process(MDP) with state space S , action space A , transition probability $P(s'|s, a)$ from state s to s' given action a and discount factor $0 < \gamma < 1$

● Problem statement

- Goal of zero-shot RL is to compute a compact representation $\mathcal{E}$ of the env by observing samples of reward-free transitions (s_t, a_t, s_{t+1}) in this env
- Once a reward function is specified later, the agent must use $\mathcal{E}$ to immediately produce a good policy, via only elementary computation without any **further planning** or **learning**
- Reward functions may be specified at test time either as a relatively small set of reward samples (s_i, r_i) , or as an explicit function $s \rightarrow r(s)$

● Successor representations (SR)

- For a finite MDP, the successor representation $M^\pi(s_0, a_0)$ of a state-action pair (s_0, a_0) under a policy π , is defined as the discounted sum of **future occurrences of each state**
 - $M^\pi(s_0, a_0, s) := \mathbb{E}[\sum_{t \geq 0} \gamma^t \mathbb{I}(s_{t+1} = s) \mid (s_0, a_0), \pi], \quad \forall s \in S$
- In matrix form, SRs can be written as $M^\pi = P \sum_{t \geq 0} \gamma^t P_\pi^t = P(I - \gamma P_\pi)^{-1}$, P_π is the state transition prob
 - M^π satisfies the matrix Bellman Equation $M^\pi = P + \gamma P_\pi M^\pi$, the Q-function can be expressed as $Q_r^\pi = M^\pi r$

● Successor features (SFs)

- Successor features extend SR to continuous MDPs by first assuming we are given a basic feature map $\varphi: S \rightarrow \mathbb{R}^d$ that embeds states into d -dimensional space, and defining the expected discounted sum of future state features
 - $\psi^\pi(s_0, a_0) := \mathbb{E}[\sum_{t \geq 0} \gamma^t \varphi(s_{t+1}) \mid s_0, a_0, \pi]$

● Successor measures (SMs)

- Successor measures extend SRs to continuous spaces by treating the distribution of future visited states as a measure M^π over the state space S
 - $M^\pi(s_0, a_0, X) := \sum_{t \geq 0} \gamma^t \Pr(s_{t+1} \in X \mid s_0, a_0, \pi), \quad \forall X \subset S, \quad \psi^\pi(s_0, a_0) = \int_S M^\pi(s_0, a_0, ds') \varphi(s')$

Previous Strategies for Zero-Shot RL

● Zero-shot RL from successor features

- Given a basic feature map $\varphi: S \rightarrow \mathbb{R}^d$ to be learned via another criterion, universal SFs learn the successor features of a particular family of policies π_z for $z \in \mathbb{R}^d$,
 - $\psi(s_0, a_0, z) = \mathbb{E}[\sum_{t \geq 0} \gamma^t \varphi(s_{t+1}) | (s_0, a_0), \pi_z]$, $\pi_z(s) := \operatorname{argmax}_a \psi(s, a, z)^T z$
- Once a reward function r is revealed, few reward samples of explicit knowledge of the function r are used to perform a linear regression of r onto the features φ
 - Namely, the $z_r := \operatorname{argmin}_z \mathbb{E}_{s \sim \rho} [(r(s) - \varphi(s)^T z)^2] = \mathbb{E}_{\rho} [\varphi \varphi^T]^{-1} \mathbb{E}_{\rho} [\varphi r] \Rightarrow$ then the policy π_z is returned
 - This policy is guaranteed to be optimal for all rewards in the linear span of the features φ
 - If $r(s) = \varphi(s)^T w$, $\forall s \in S$, then $z_r = w$, and π_{z_r} is the optimal policy for reward r

● Zero-shot RL from Forward-Backward representation (FB)

- Forward-backward representation look for $F: S \times A \times \mathbb{R}^d \rightarrow \mathbb{R}^d$ and $B: S \rightarrow \mathbb{R}^d$ such that the long-term transition prob M^{π_z} decompose as
 - $M^{\pi_z}(s_0, a_0, ds') \approx \mathbf{F}(s_0, a_0, z)^T \mathbf{B}(s') \rho(ds')$, $\pi_z(s) := \operatorname{argmax}_a F(s, a, z)^T z$
 - In a finite space, M^{π_z} could be decomposed as $M^{\pi_z} = F_z^T B \operatorname{diag}(\rho)$
- Once a reward function r is revealed, $z_r := \mathbb{E}_{s \sim \rho} [r(s) B(s)]$ from a few reward samples or from explicit knowledge of the function r are estimated (e.g. $z_r = B(s)$ to reach s)
 - Then the policy π_{z_r} is returned.
 - Any reward function r , the policy π_{z_r} is optimal for r , with optimal Q-function $Q_r^* = F(s, a, z_r)^T z_r$

Algorithms for SF and FB Representations (I)

- The authors suggested the novel losses used to train ψ in SFs, and F, B in FB
 - To obtain a full zero-shot RL algorithm, SFs must specify the basic feature φ , thus they proposed the ten possible choices based on existing or new representations for RL
- Learning the SF $\psi^T z$ instead of ψ
 - The successor feature ψ satisfy the Bellman equation $\psi^\pi = P\varphi + \gamma P_\pi \psi^\pi$, the collection of ordinary Bellman Eq for each component of φ in BBQ-network
 - Therefore $\psi(s, a, z)$ for each z could be trained by minimizing the Bellman residuals as follows,
 - $\left\| \psi(s_t, a_t, z) - \varphi(s_{t+1}) - \gamma \bar{\psi}(s_{t+1}, \pi_z(s_{t+1}), z) \right\|^2$, z is random sampled by logic
 - They proposed the novel loss instead of the vector-valued Bellman residual above,
 - $\mathcal{L}(\psi) := \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \rho} \left[\left((\psi(s_t, a_t, z))^T z - \varphi(s_{t+1})^T z - \gamma \bar{\psi}(s_{t+1}, \pi_z(s_{t+1}), z)^T z \right)^2 \right]$ for each z
 - This trains $\psi(\cdot, z)^T z$ as the Q-function of reward $\varphi^T z$, the only case needed, while training the full vector $\psi(\cdot, z)$ amounts to training the Q-functions of each policy π_z for all rewards $\varphi^T z'$ for all $z' \in \mathbb{R}^d$ including $z' \neq z$.

● Learning the FB representations: the FB training loss

- The successor measure M^π satisfies a Bellman-like equation, $M^\pi = P + \gamma P_\pi M^\pi$, as matrices in the finite case and as measures in the general case in [Blier et al]
 - For any policy π_z , the Q-function for the reward r can be written $Q_r^{\pi_z} = M^{\pi_z} r$ in matrix form.
 - This is equal to $F_z^T B \text{diag}(\rho) r$; thus assume that the $z_r := B \text{diag}(\rho) r = \mathbb{E}_{s \sim \rho}[B(s)r(s)]$, the Q-function is obtained as $Q_r^{\pi_z} = F_z^T z_r$ for any $z \in \mathbb{R}^d$
- FB could be learned by iteratively minimizing the Bellman residual on the parametric model $M = F^T B \rho$.
 - Using a suitable norm $\|\cdot\|_\rho$ for the bellman residual leads to a loss expressed as expectation from the dataset
 - $\mathcal{L}(F, B) := \left\| F_z^T B \rho - (P + \gamma P_{\pi_z} \bar{F}_z^T \bar{B} \rho) \right\|$
 $= \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \rho, s' \sim \rho} \left[\left(F(s_t, a_t, z)^T B(s') - \gamma \bar{F}(s_{t+1}, \pi_z(s_{t+1}), z)^T \bar{B}(s') \right)^2 \right] - 2 \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \rho} [F(s_t, a_t, z)^T B(s_{t+1})] + \text{Const}$
 - z is random sampled by logic
- The authors proposed that the last term involves $B(s_{t+1})$ instead of $B(s_t)$, because they used s_{t+1} instead of s_t for the successor measure

● Learning basic features φ for SF

- Any representation learning method could be used to supply φ
 - The authors suggested the 10 basic features and described the precise learning objective for each
- 1. Random Feature (Rand)
 - Using a non-trainable randomly initialized network as features
- 2. Autoencoder (AEnc)
 - Learning a decoder $f: \mathbb{R}^d \rightarrow S$ to recover the state from its representation φ
 - $\min_{f, \varphi} \mathbb{E}_{s \sim \mathcal{D}} [(f(\varphi(s)) - s)^2]$
- 3. Inverse Curiosity Module (ICM)
 - Aiming at extracting the controllable aspects of the environment
 - Training an inverse dynamics model $g: \mathbb{R}^d \times \mathbb{R}^d \rightarrow A$ to predict the action used for a transition between two consecutive states

$$- \min_{g, \varphi} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} \left[\|g(\varphi(s_t), \varphi(s_{t+1})) - a_t\|^2 \right]$$

● Learning basic features φ for SF

■ 4. Transition model (Trans)

- Learning the one-step dynamics $f: \mathbb{R}^d \times A \rightarrow S$ that predicts the next state from the current state representation
 - $\min_{f, \varphi} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [(f(\varphi(s_t), a_t) - s_{t+1})^2]$

■ 5. Latent transition model (Latent)

- Learning the latent dynamics model but instead of predicting the next state, it predicts its representation
 - $\min_{f, \varphi} \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [(f(\varphi(s_t), a_t) - \varphi(s_{t+1}))^2]$

■ 6. Laplacian Eigenfunction (Lap)

- Wu et al consider the symmetrized MDP graph Laplacian induced by an exploratory policy π , defined as $\mathcal{L} = I - \frac{1}{2} (P_\pi \text{diag}(\rho)^{-1} + \text{diag}(\rho)^{-1} (P_\pi)^T)$
- They propose to learn the eigenfunctions of $\mathcal{L}$ via the spectral graph drawing objective as follows:
 - $\min_{\varphi} \mathbb{E}_{(s_t, s_{t+1}) \sim \mathcal{D}} [\|\varphi(s_t) - \varphi(s_{t+1})\|^2] + \lambda \mathbb{E}_{s \sim \mathcal{D}, s' \sim \mathcal{D}} [(\varphi(s)^T \varphi(s'))^2 - \|\varphi(s)\|_2^2 - \|\varphi(s')\|_2^2]$
 - Where the second term is an orthonormality regularization to ensure that $\mathbb{E}_{s \sim \rho} [\varphi(s) \varphi(s)^T] \approx I$

Algorithms for SF and FB representations (V)

● Learning basic features φ for SF

■ 7. Low-Rank Approximation of P

- Learning the features by estimating a low-rank model of the transition probability densities: $P(ds'|s, a) \approx \mathcal{X}(s, a)^T \mu(s') \rho(ds')$.
- The corresponding loss on $\mathcal{X}^T \mu - P/\rho$ could be expressed as

$$\begin{aligned} & - \min_{\mathcal{X}, \mu} \mathbb{E}_{(s_t, a_t) \sim \rho, s' \sim \rho} \left[\left(\mathcal{X}(s_t, a_t)^T \mu(s') - \frac{P(ds'|s_t, a_t)}{\rho(ds')} \right)^2 \right] = \mathbb{E}_{(s_t, a_t) \sim \rho} \left[\left(\mathcal{X}(s_t, a_t)^T \mu(s') \right)^2 \right] - 2 \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \rho} [\mathcal{X}(s_t, a_t)^T \mu(s_{t+1})] + C \\ & - \text{This loss is also a special case of the FB loss by setting } \gamma = 0, \text{ omitting } z \end{aligned}$$

■ 8. Contrastive Learning

- Learning the representations by pushing positive pairs closer together while keeping negative pairs part
 - Here, two states are considered similar if they lie close on the same trajectory
- They proposed SimCLR-like objective as

$$- \min_{\mathcal{X}, \varphi} - \mathbb{E}_{k \sim \text{Geom}(1-\gamma_{CL})} \left[\log \frac{\exp(\text{cosine}(\mathcal{X}(s_t), \varphi(s_{t+k})))}{\mathbb{E}_{s' \sim \mathcal{D}} \exp(\text{cosine}(\mathcal{X}(s_t), \varphi(s'))) \right], \quad \text{cosine}(u, v) = \frac{u^T v}{\|u\|_2 \|v\|_2}$$

■ 9. Low-Rank Approximation of SR

- Learning the features by estimating a low-rank model of the successor measure for exploration policy

$$- \min_{\mathcal{X}, \varphi} \mathbb{E}_{(s_t, s_{t+1}) \sim \mathcal{D}} \left[\left(\mathcal{X}(s_t)^T \varphi(s') - \gamma \bar{\mathcal{X}}(s_{t+1}) \bar{\varphi}(s') \right)^2 \right] - 2 \mathbb{E}_{(s_t, s_{t+1}) \sim \mathcal{D}} [\mathcal{X}(s_t)^T \varphi(s_{t+1})], \quad \text{where } \bar{\mathcal{X}} \text{ and } \bar{\varphi} \text{ are target version of } \mathcal{X} \text{ and } \varphi$$

Experiments

Environments

- All the methods were tested in DeepMind Control Suite(ExORL Benchmarks)

- Tasks and environments

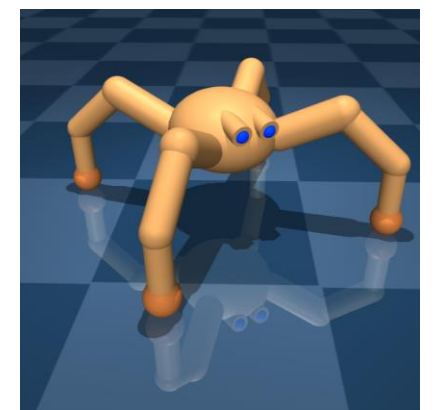
- Point-mass Maze
 - State, action: 4/2 dim vectors
- Walker: a planner walker
 - State, action: 24/6 dim vectors
- Cheetah: a running planar biped
 - State, action: 17/6 dim vectors
- Quadraped: a four-leg ant navigating in 3D space
 - State, action: 78/12 dim vectors

- Replay buffers

- RND: 10M training transition with RND
- APS: 10M training transition with APS
- Proto: 10M training transition with ProtoRL



Point-mass Maze / Walker

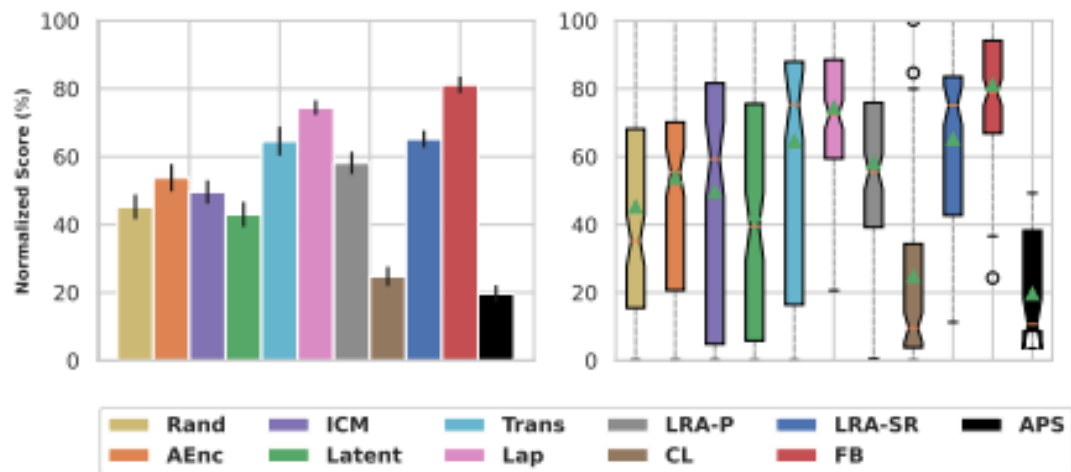


Cheetah / Quadraped

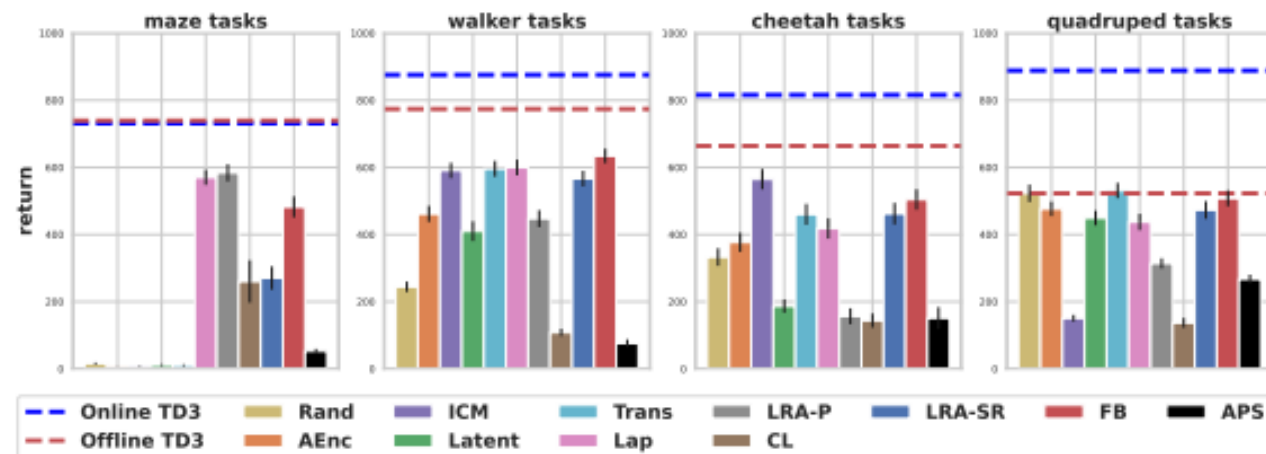
Results – Zero-shot performance of proposed methods

● Comparison with 11 methods(FB and 10 SF-based models) and offline/online TD3

- The performance of each method for each task in each env, averaged over the three buffers and 10 seeds
- Control group
 - Online TD3: with task reward, and free environment interactions
 - Offline TD3: with task reward, and training from the replay buffer
- FB and Lap show superior performance than other methods
 - FB and LaP reached about 83% and 76% of supervised offline TD3 performance



Average plots of zero-shot results(task, env, buffer, seeds)



Average scores over tasks for each env

Thank you!

Q&A